

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 02 -

CONTROL PLANE E WORKER NODES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

O QUE VEM POR AÍ?

Nesta aula, vamos analisar em detalhes a arquitetura fundamental do Kubernetes, explorando em detalhes minuciosos o funcionamento do Control Plane e dos Worker Nodes. Esta é uma das aulas mais críticas para compreender como o Kubernetes opera internamente, pois estabelece as bases para todos os conceitos avançados que serão abordados ao longo do curso.

Serão analisados não apenas os componentes principais, como API Server, etcd, Scheduler e Controller Manager, mas também suas interações complexas, algoritmos de decisão, mecanismos de failover e como cada peça se integra para formar um sistema distribuído robusto e escalável. Nos Worker Nodes, exploraremos o kubelet, kube-proxy e container runtime em profundidade, incluindo suas configurações avançadas, otimizações de performance e troubleshooting.

O objetivo é proporcionar uma compreensão tão sólida que você seja capaz de diagnosticar problemas complexos em produção, otimizar clusters para diferentes workloads, implementar configurações de alta disponibilidade e entender as decisões arquiteturais por trás do Kubernetes.

HANDS ON

Nessas videoaulas, teremos exercícios passo a passo de instalação e configuração dos principais componentes que compõem um cluster Kubernetes.



SAIBA MAIS

Fundamentos Arquiteturais do Kubernetes

A Filosofia da Arquitetura Distribuída

O Kubernetes foi projetado seguindo princípios fundamentais de sistemas distribuídos que garantem sua robustez e escalabilidade. A separação clara entre Control Plane e Worker Nodes não é apenas uma decisão de design, mas uma necessidade arquitetural que permite:

Separação de Responsabilidades: o Control Plane se concentra nas decisões estratégicas (onde executar, como escalar, quando recuperar), enquanto os Worker Nodes focam na execução tática (executar containers, manter rede, coletar métricas).

Escalabilidade Horizontal: novos Worker Nodes podem ser adicionados sem impactar o Control Plane, permitindo crescimento linear da capacidade de processamento.

Resiliência por Design: a falha de um Worker Node não afeta outros nós nem o Control Plane, garantindo isolamento de falhas e recuperação automática.

Flexibilidade de Deployment: o Control Plane pode ser executado em infraestrutura dedicada e altamente disponível, enquanto Worker Nodes podem usar hardware commodity.

Padrões de Comunicação e Protocolos

O Kubernetes implementa um modelo de comunicação baseado em APIs RESTful sobre HTTP/HTTPS, mas com nuances importantes:

API-First Architecture: toda interação no cluster passa pela API do Kubernetes, garantindo consistência, auditoria e controle de acesso centralizado.

Watch Pattern: componentes utilizam o padrão "watch" para receber notificações em tempo real sobre mudanças de estado, reduzindo latência e uso de recursos.

Declarative State Management: o sistema trabalha com estado declarativo, onde você especifica o que quer (desired state) e o Kubernetes trabalha continuamente para alcançar esse estado.

Control Plane - O Cérebro do Kubernetes

Visão Geral Arquitetural

O Control Plane do Kubernetes é um sistema distribuído complexo que implementa o padrão "control loop" em múltiplas camadas. Cada componente opera de forma independente, mas coordenada, criando um sistema auto-healing e auto-scaling.

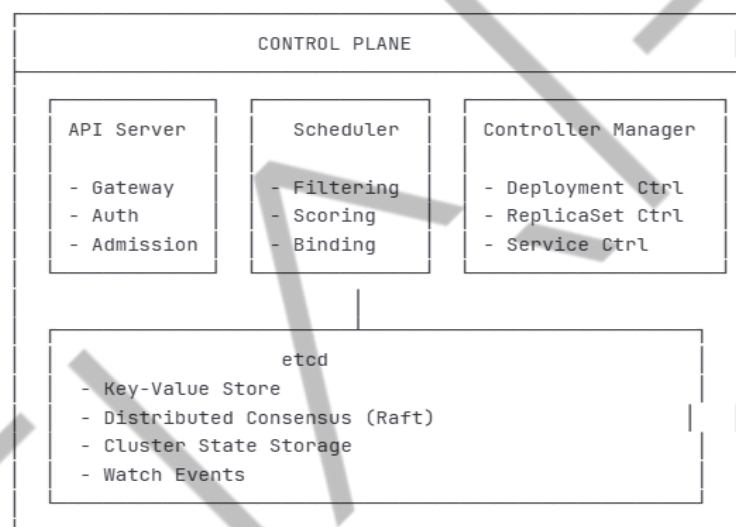


Figura 1 – Control Plane
Fonte: Elaborado pelo autor (2025)

API Server - O Gateway Inteligente: o API Server é muito mais do que um simples gateway HTTP. É um servidor altamente sofisticado que implementa múltiplas funcionalidades críticas:

Arquitetura Interna do API Server

Request Pipeline: cada requisição passa por uma pipeline de processamento rigorosamente definida:

1. **Authentication:** verifica a identidade do requisitante usando certificados X.509, tokens JWT, ou plugins externos.

2. **Authorization:** determina se o usuário tem permissão para realizar a operação usando RBAC, ABAC ou webhooks.
3. **Admission Control:** aplica políticas de negócio e validações antes de persistir o objeto.
4. **Validation:** verifica se o objeto está sintática e semanticamente correto.
5. **Persistence:** armazena o objeto no etcd.
6. **Response:** retorna confirmação ou erro para o cliente.

Funcionalidades Avançadas

Custom Resource Definitions (CRDs): o API Server permite extensão dinâmica do schema da API, possibilitando que pessoas desenvolvedoras criem novos tipos de recursos sem modificar o core do Kubernetes.

Aggregated API Servers: permite que API servers externos se registrem como extensões, criando uma experiência unificada para recursos customizados complexos.

OpenAPI/Swagger Integration: gera documentação automática da API e permite validação client-side, facilitando o desenvolvimento de ferramentas.

Webhook Integration: suporta webhooks para admission control, authentication e authorization, permitindo integração com sistemas externos.

etcd - O Sistema Nervoso Central

O etcd é um banco de dados distribuído que utiliza o algoritmo de consenso Raft para garantir consistência em um cluster. É o único local onde o Kubernetes armazena seu estado.

Arquitetura e Funcionamento do Raft

O protocolo Raft garante que todas as operações sejam aplicadas na mesma ordem em todos os nós do cluster:

Leader Election: um nó é eleito leader e é responsável por todas as escritas
Log Replication: O leader replica todas as mudanças para os followers
Safety: Garante que logs commitados nunca sejam perdidos.

Estrutura de Dados no etcd

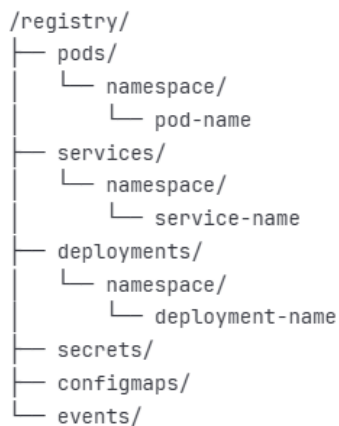


Figura 2 - Estrutura de Dados no etcd
Fonte: Elaborado pelo autor (2025)

Scheduler - O Estrategista de Recursos: o Kubernetes Scheduler é um componente sofisticado que toma decisões de placement usando algoritmos complexos de otimização.

Processo de Scheduling Detalhado: o processo de scheduling acontece em duas fases principais:

Fase 1 - Filtering (Predicates):

- **NodeResourcesFit:** verifica se o nó tem recursos suficientes (CPU, memória, storage).
- **NodeAffinity:** aplica regras de afinidade de nó.
- **PodAffinity/AntiAffinity:** considera afinidade entre pods.
- **Taints and Tolerations:** respeita restrições de scheduling.
- **VolumeBinding:** verifica disponibilidade de volumes.

Fase 2 - Scoring (Priorities):

- **NodeResourcesFit:** pontua nós com base em utilização de recursos.
- **ImageLocality:** favorece nós que já têm a imagem do container.
- **InterPodAffinity:** pontua baseado em afinidade entre pods.

- **NodeAffinity:** aplica pesos às preferências de afinidade.

Controller Manager - O Maestro dos Control Loops: o Controller Manager é uma coleção de controllers que implementam a lógica de negócio do Kubernetes. Cada controller executa um "control loop" independente.

Worker Nodes - Os Executores Especializados

Arquitetura Detalhada dos Worker Nodes: cada Worker Node é um ambiente de execução completo que deve ser capaz de:

- Executar containers de forma isolada e segura;
- Manter comunicação com o Control Plane;
- Implementar políticas de rede;
- Coletar métricas e logs;
- Gerenciar recursos locais (CPU, memória, storage).

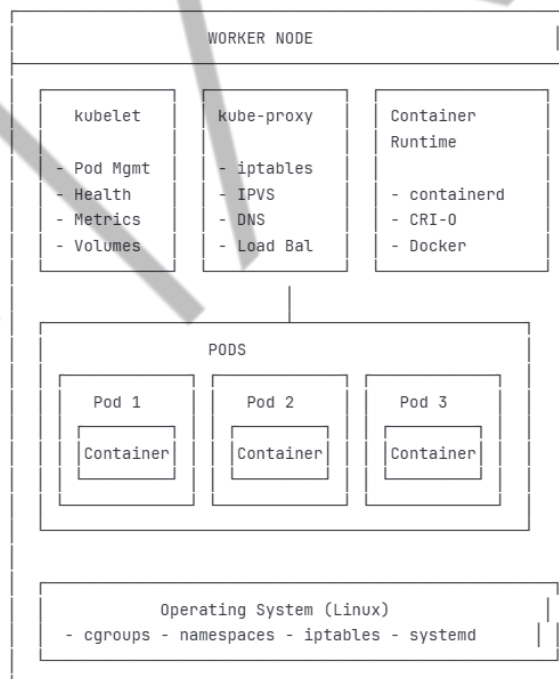


Figura 3 - Worker Nodes
Fonte: Elaborado pelo autor (2025)

kubelet - O Agente Node Superinteligente: o kubelet é o agente mais complexo do Kubernetes, responsável por traduzir especificações declarativas em ações concretas no sistema operacional.

kube-proxy - O Maestro da Rede: o kube-proxy implementa a abstração de Services do Kubernetes, fornecendo load balancing e service discovery.

- Comunicação e Fluxos de Dados Avançados;
- Fluxos Críticos de Operação;
- Criação Completa de um Pod;
- Alta Disponibilidade e Topologias de Cluster;
- Control Plane HA - Arquitetura Stacked etcd.

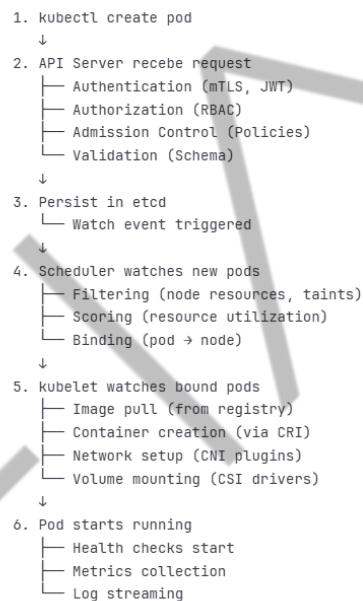


Figura 4 – Kubelet
Fonte: Elaborado pelo autor (2025)

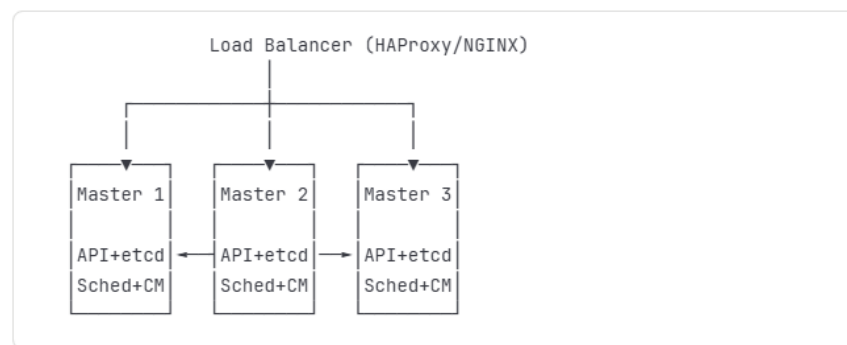


Figura 5 – Load Balancer
Fonte: Elaborado pelo autor (2025)

- Troubleshooting e Diagnósticos Avançados;
- Problemas Críticos do Control Plane;
- API Server Troubleshooting;
- Sintoma: API Server não responde ou responde lentamente.

Soluções Comuns:

- Aumentar limites de file descriptors;
- Otimizar configurações de timeout;
- Verificar e renovar certificados;
- Analisar admission controllers demorados.

MERCADO, CASES E TENDÊNCIAS

Netflix - Global Scale Architecture

Desafio: gerenciar milhares de microsserviços em múltiplas regiões AWS

Solução:

- Clusters Kubernetes regionais com 1000+ nós cada;
- Control Plane dedicado com etcd distribuído;
- Custom controllers para integração com sistemas internos;
- Chaos engineering com Chaos Monkey para Kubernetes.

Spotify - Developer Platform

Desafio: suportar 300+ equipes de desenvolvimento independentes

Solução:

- Kubernetes como plataforma interna (Backstage + K8s);
- Namespace per team com resource quotas;
- GitOps workflow com ArgoCD;
- Service mesh para observabilidade entre serviços.

Uber - Real-time Workloads

Desafio: processar milhões de requests por segundo com baixa latência

Solução:

- Clusters especializados por workload type;
- Custom schedulers para latency-sensitive workloads;
- Resource isolation com cpu-manager e topology-manager;
- Real-time monitoring com Prometheus + custom metrics.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula exploramos os principais elementos que compõem a arquitetura do Kubernetes, destacando a divisão entre Control Plane e Worker Nodes. Foram detalhados os serviços do Control Plane — como API Server, etcd, Scheduler e Controller Manager — e os componentes dos Worker Nodes, incluindo kubelet, kube-proxy e o container runtime.

Os conceitos apresentados fornecem a base para entender como o Kubernetes consegue orquestrar aplicações em contêineres de forma eficiente, escalável e resiliente em cenários de produção.

REFERÊNCIAS

Cluster Architecture. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/concepts/overview/components/>. Acesso em: 25 set. 2025.

Control Plane Components. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/concepts/overview/components/>. Acesso em: 25 set. 2025.

Etcd. Clustering Guide. Disponível em: <https://etcd.io/docs/v3.5/op-guide/clustering/>. Acesso em: 25 set. 2025.

GITHUB. **Kubernetes The Hard Way**. Disponível em: <https://github.com/kelseyhightower/kubernetes-the-hard-way>. Acesso em: 25 set. 2025.

Node Components. **Kubernetes Documentation**. Disponível em: <https://kubernetes.io/docs/concepts/architecture/nodes/>. Acesso em: 25 set. 2025.

PALAVRAS-CHAVE

Control Plane. Worker Nodes. API Server.

EMSE



POSTECH